

Recurrences

Topics

- Introduction to recurrences
- Applications of recurrence relations
- Solving recurrences

1

Recurrences

Definition

A **recurrence** is a **relation or equation** that describes a **function in terms of its lower order arguments**, with the following characteristics

- (i) The function is defined over a set of **natural numbers**
- (ii) The definition includes a **base** value for the function, called **boundary condition**

Example(1): The factorial function $f(n)=n!$ can be expressed
By the recurrence

$$\begin{aligned} f(n) &= n \cdot f(n-1) \\ f(0) &= 1 \quad \text{(boundary condition)} \end{aligned}$$

Example(2) The **Fibonacci Sequence** $f(n)$ is usually defined as

$$\begin{aligned} f(n) &= f(n-1) + f(n-2) \\ f(0) &= 0, \quad f(1) = 1 \quad \text{(boundary condition)} \end{aligned}$$

2

Recurrences

Taxonomy

Recurrences are frequently used to specify *running times (complexity)* for a broad categories of problems. Common examples are *decrease-and-conquer*, *divide-and-conquer*, and *probabilistic analysis* algorithms.

- In a *decrease-and-conquer algorithm* the size of a problem is reduced by a *constant* amount in each iteration. The running time can be specified as

$$T(n) = T(n-c) + f(n),$$

where c is the *reduction constant*, and $f(n)$ is the cost of reduction in each iteration.

- In a *divide-and-conquer* algorithm a problem is split (*divided*) into subproblems, which are then *solved (conquered)* and *combined* to obtain solution to the main problem. In general the running time is expressed as

$$T(n) = aT(n/b) + f(n)$$

where a is the number of subproblems and b is the size of each subproblem. The function $f(n)$ is the cost of *dividing* and *combing* the subproblem solutions.

- In *probabilistic analysis* algorithm, the average running time is typically written as

$$T(n) = a_{n-1}T(n-1) + a_{n-2}T(n-2) + \dots + a_1T(1) + f(n)$$

3

Recurrences

Examples

The following examples demonstrate the use of recurrence for the running times of common algorithms. Here $T(n)$ denotes the running time for a problem of size n .

Example(1): Here is an example of recurrence relation for *decrease- and-conquer* problem.

$$T(n) = T(n-1) + cn$$

Subproblem size
Cost of decreasing

Example(2) This example illustrates the recurrence relation for *divide-and-conquer* problem.

$$T(n) = 8T(n/4) + cn^2$$

Number of subproblems
Subproblem size
Cost of dividing and combining

4

Recurrences

Applications

Some typical applications of recurrences are as follows;

- *Linear Search*: $T(n) = T(n-1) + c$
- *Selection Sort*: $T(n) = T(n-1) + n$
- *Binary Search*: $T(n) = T(n/2) + c$
- *Quick /Merge Sort*: $T(n) = 2T(n/2) + n$
- *Finding Maximum*: $T(n) = 2T(n/2) + c$
- *Multiplying Integers*: $T(n) = 3T(n/2) + n$
- *Multiplying Matrices*: $T(n) = 8T(n/2) + n^2$
- *Quick Sort (Expected)*: $T(n) = \left(\sum_{k=1}^{n-1} T(n-k) \right) / n + n$

5

Recurrence Solution

Types

The solution of a recurrence is expressed in a **closed form** or in **asymptotic notation**.

Closed Form: The closed form provides an exact solution of recurrence equation. It is usually expressed as a formula. The closed form solution to the recurrence

$$T(n) = \begin{cases} 1, & n=2 \\ 2T(n/2) + 2 & n > 2 \end{cases}$$

is given by the formula $T(n) = (3/2)n - 2$.

Asymptotic Notation: For most problems the exact solution for a recurrence cannot be determined or it does not exist. For analysis purpose, the solution is often expressed in asymptotic notation (O , θ , Ω) For example, the solution to the recurrence

$$T(n) = \begin{cases} c, & n=1 \\ 3T(n/4) + cn^2 & n > 1 \end{cases}$$

is expressed as $T(n) = \theta(n^2)$

6

Solving Recurrences

Methods

Common methods for the solution are of recurrences are:

- *The Iterative Method*
- *The Substitution Method*
- *The Recursion Tree Method*
- *Using Theorems*
- *Using Induction Method*
- *Solving Characteristic Equation*

7

The Iteration Method

Procedure

In the **iteration method** the recurrence is solved by following the **top-down approach**. It involves following steps:

- (1) Using definition, equations are set up for arguments $n, n-1, n-2, \dots$
- (2) On reaching the bottom level the boundary condition is applied.
- (3) The equations are summed up.
- (4) Finally, the solution is obtained by canceling out identical terms on the left-hand and right-hand sides of the iterated equations

- The iteration method is particularly useful in solving **decrease-and-conquer** problems. In other cases additional efforts are required to cancel out the terms appearing on both sides of the final equation

8

The Iteration Method

Example(1): Here is a recurrence for the linear search. It is based on decrease-and-conquer algorithm:

$$T(0)=0$$

$$T(n)= T(n-1) +c$$

Iterating the recurrence:.

$$T(n) = T(n-1) + c$$

$$T(n-1) = T(n-2) + c$$

$$T(n-2) = T(n-3) + c$$

.....

$$T(3) = T(2) + c$$

$$T(2) = T(1) + c$$

$$T(1) = T(0) + c$$

Adding both sides of the equations, and canceling equal terms:

$$T(n) = c + c + \dots + c$$

$$\text{Or, } T(n)=n.c$$

It follows that $T(n)=\theta(n)$

9

The Iteration Method

Example(2): In *selection sort*, the largest element in an array is searched. It is exchanged with the last element in the array. This procedure is repeatedly applied to sub-arrays.

The recurrence for selection sort algorithm is as follows:

$$T(0)=0$$

$$\underbrace{T(n)}_{\text{Sorting } n \text{ elements}} = \underbrace{T(n-1)}_{\text{Sorting } n-1 \text{ elements}} + \underbrace{c.n}_{\text{Finding maximum and exchanging}}$$

Iterating the recurrence:

$$T(n) = T(n-1) + c.n$$

$$T(n-1) = T(n-2) + c.(n-1)$$

$$T(n-2) = T(n-3) + c.(n-2)$$

.....

$$T(3) = T(2) + c.3$$

$$T(2) = T(1) + c.2$$

$$T(1) = T(0) + c.1$$

Adding both sides of the equations, and canceling equal terms:

$$T(n) = c(1 + 2 + 3 + \dots + n)$$

Summing the arithmetic series:

$$T(n)=c.n(n+1)/2$$

It follows that

$$T(n)=\theta(n^2)$$

10

The substitution Method

Procedure

The **substitution method** provides a systematic procedure for solving recursive equation. It is similar to the iteration method because it also follows the top-down approach. The procedure involves following steps:

- (1) In the recurrence, values are plugged in repeatedly on the right-hand side of the equation.
 - (2) The procedure is repeated until the **base case** is reached
 - (3) The iterative steps generate some kind of **pattern** or a **series**.
 - (4) The summation for the series is analyzed to determine the asymptotic behavior.
- The method is useful for solving both decrease-and-conquer and divide-and-conquer Problems.
 - To facilitate the step for determining the values to be plugged in, a **copy** of the recurrence is usually maintained for reference.

11

Substitution Method

Example(1): The recurrence for the binary search algorithm is

$$T(1) = c$$

$$T(n) = T(n/2) + c, \quad n > 1$$

$$1^{st} \text{ substitution, } T(n) = T(n/2) + c = T(n/2^1) + c$$

$$2^{nd} \text{ substitution, } T(n) = T(n/4) + 2c = T(n/2^2) + 2.c$$

$$3^{rd} \text{ substitution, } T(n) = T(n/8) + 3c = T(n/2^3) + 3.c$$

$$\dots\dots\dots k^{th} \text{ substitution, } T(n) = T(n/2^k) + k.c$$

It will be seen that, on continuing, the base case $T(1)$ is reached when $n / 2^k = 1$, or $n = 2^k$, i.e. $k = \lg n$

Substituting for k , we get

$$\begin{aligned} T(n) &= T(1) + \lg n . c \\ &= c + \lg n . c \end{aligned}$$

Thus, $T(n) = \theta(\lg n)$

12

Substitution Method

Example(2): It can be shown that algorithm for finding the largest element in an array can be expressed as recurrence:

$$\begin{aligned} T(1) &= c \\ T(n) &= 2T(n/2) + c, \quad n > 1 \end{aligned}$$

Initially: $T(n) = 2.T(n/2) + c = 2.T(n/2^1) + 2^0 c$

Substituting for $T(n/2)$: $T(n) = 2.[2.T(n/4) + c] + c$
 $= 4.T(n/4) + 3c = 2^2 T(n/2^2) + (2^0 + 2^1).c$

Substituting for $T(n/4)$: $T(n) = 4.[2.T(n/8) + c] + 3c$
 $= 8.T(n/8) + 7.c = 2^3 T(n/2^3) + (2^0 + 2^1 + 2^2).c$

.....
 After k^{th} substitution, $T(n) = 2^k T(n/2^k) + (2^0 + 2^1 + 2^2 + \dots + 2^{k-1}).c$
 $= 2^k T(n/2^k) + (2^k - 1)c$

Continuing, it will be seen that the base case $T(1)$ is reached when $n/2^k = 1$, or $n = 2^k$,
 Substituting for 2^k , we get

$$\begin{aligned} T(n) &= n.T(1) + (n-1).c = n.c + n.c - c \\ T(n) &= 2n.c - c \\ T(n) &= \theta(n) \end{aligned}$$

13

Using Substitution Method

Example(3): $T(1) = c$
 $T(n) = 2T(n/2) + cn, \quad n > 1$

Using substitution method it can be shown that after k^{th} substitution:

$$T(n) = cnk + 2^k T(n/2^k)$$

The base case $T(1)$ is reached when $n/2^k = 1$, or $k = \lg(n)$

Therefore, $T(n) = \theta(n \lg n)$

Example(4): $T(1) = c$
 $T(n) = 3T(n/4) + cn, \quad n > 1$

Using substitutions it can be shown that

$$T(n) = cn[(3/4)^0 + (3/4)^1 + (3/4)^2 + \dots + (3/4)^{k-1}] + 3^k T(n/4^k), \text{ after } k^{th} \text{ iteration}$$

The base case, $T(1)$, is reached when $n/4^k = 1$, or $k = \log_4(n)$

The contribution by the decreasing geometric series is $\theta(1)$.

$$T(n) = cn\theta(1) + 3^{\log_4 n} = \theta(n) + n^{\log_4 3} = \theta(n)$$

14

Using Theorems

15

Using Theorems

Theorem#1

If a is the number of subproblems and b the size of each subproblem, then the solution to the **divide-and-conquer** recurrence

$$\begin{aligned} T(1) &= c \\ T(n) &= a T(n/b) + \Theta(n), \quad \text{for } n > 1 \end{aligned}$$

is given by

$$T(n) = \begin{cases} \theta(n), & \text{when } a < b \\ \theta(n \log n), & \text{when } a = b \\ \theta(n^{\log_b a}), & \text{when } a > b \end{cases}$$

- The theorem provides solution to problems in which cost of dividing and combining subproblems increases **linearly**.

16

Using Theorem #1

According to **Theorem#1**, the solution of recurrence $T(n) = aT(n/b) + cn$ is given by

$$T(n) = \begin{cases} \theta(n) & \text{when } a < b \quad (\text{case 1}) \\ \theta(n \lg n) & \text{when } a = b \quad (\text{case 2}) \\ \theta(n^{\log_b a}) & \text{when } a > b \quad (\text{case 3}) \end{cases}$$

Example(1): $T(n) = 2T(n/4) + cn$

Here, $a=2$, $b=4$. Since $a < b$, case 1 applies

Therefore, $T(n) = \theta(n)$

Example(2): $T(n) = 2T(n/2) + cn$

$a=2$, $b=2$ Since $a = b$, case 2 applies

Therefore, $T(n) = \theta(n \lg n)$

Example(3): $T(n) = 3T(n/2) + cn$

$a=3$, $b=2$ Since $a > b$, case 3 applies

Therefore, $T(n) = \theta(n^{\log_2 3}) = \theta(n^{\lg 3})$

17

Theorem # 1

Proof

The **method of substitution** can be used to obtain solution to the recurrence

$$T(n) = aT(n/b) + cn$$

➤ The solution is as follows. Substituting for $T(n/b)$:

$$T(n) = a[aT(n/b^2) + c(n/b)] + cn = a^2T(n/b^2) + cn[1 + (a/b)]$$

Substituting for $T(n/b^2)$

$$T(n) = a^2[aT(n/b^3) + c(n/b^2)] + cn[1 + (a/b)] = a^3T(n/b^3) + cn[1 + (a/b) + (a/b)^2]$$

Substituting for $T(n/b^3)$:

$$T(n) = a^3[aT(n/b^4) + c(n/b^3)] + cn[1 + (a/b) + (a/b)^2] = a^4T(n/b^4) + cn[1 + (a/b) + (a/b)^2 + (a/b)^3]$$

Continuing, after k th substitution,

$$T(n) = a^kT(n/b^k) + cn[1 + (a/b) + (a/b)^2 + (a/b)^3 + \dots + (a/b)^{k-1}]$$

The term $T(n/b^k)$ would reduce to base case $T(1)$ when $n/b^k = 1$, or $b^k = n$.

Taking logarithm with base b , $k = \log_b n$. Substituting for k in the summation:

$$T(n) = a^{\log_b n} T(1) + cn[1 + (a/b) + (a/b)^2 + (a/b)^3 + \dots + (a/b)^{\log_b n - 1}]$$

$$= c \cdot a^{\log_b n} + cn[1 + (a/b) + (a/b)^2 + (a/b)^3 + \dots + (a/b)^{\log_b n - 1}]$$

Cont'd

18

Theorem#1

Proof

$$T(n) = c \cdot a^{\log_b n} + cn[1 + (a/b) + (a/b)^2 + (a/b)^3 + \dots + (a/b)^{\log_b n - 1}] \dots \dots \dots (1)$$

Now, consider $a^{\log_b n}$. Multiplying and dividing with $b^{\log_b n}$.

$$a^{\log_b n} = \frac{a^{\log_b n} \cdot b^{\log_b n}}{b^{\log_b n}} = (a/b)^{\log_b n} \cdot b^{\log_b n}$$

Since, $b^{\log_b n} = n^{\log_b b} = n$, it follows, $a^{\log_b n} = n(a/b)^{\log_b n}$

Substituting into the above equation (1), and rearranging

$$T(n) = cn[1 + (a/b) + (a/b)^2 + (a/b)^3 + \dots + (a/b)^{\log_b n}] \dots \dots \dots (2)$$

▪ The right side is **geometric series with geometric ratio a/b** . The asymptotic behavior is determined by the

largest term of the series. Three cases need to be considered.

Case $a < b$: The geometric series (2) has decreasing terms. Its first term, $(a/b)^0 = 1$, makes the main contribution asymptotically, which is $\theta(1)$. Therefore, recurrence has the solution

$$T(n) = cn \cdot \theta(1) = \theta(n)$$

Case $a = b$: All terms of the series make equal contribution of 1. There being $\log_b n$ terms,

$$T(n) = cn \cdot \log_b n = \theta(n \log_b n)$$

Case $a > b$: The geometric series has increasing terms. The last term, $(a/b)^{\log_b n}$, makes main contribution asymptotically

Since, $(a/b)^{\log_b n} = a^{\log_b n} / b^{\log_b n} = n^{\log_b a} / n^{\log_b b} = n^{\log_b a} / n$

Therefore, the recurrence has the solution

$$T(n) = cn \cdot n^{\log_b a} / n = c n^{\log_b a} = \theta(n^{\log_b a})$$

19

Using Theorems

Theorem #2

:If a is the number of subproblems and b the size of each subproblem, then the solution to the **divide-and-conquer** recurrence

$$T(1) = c$$

$$T(n) = a T(n/b) + c n^x \quad \text{for } n > 1, \text{ where } x \text{ is some constant,}$$

is given by

$$T(n) = \begin{cases} \theta(n^x), & \text{when } a < b^x \\ \theta(n^x \log n), & \text{when } a = b^x \\ \theta(n^{\log_b a}), & \text{when } a > b^x \end{cases}$$

▪ The theorem provides solution to problems in which cost of dividing and combining subproblems is some **polynomial** term..

▪ The recurrence can be solved by substitution method.

20

Using Theorem #2

According to **Theorem#2**, the solution of recurrence $T(n)=aT(n/b) + cn^x$, where x is some positive constant is given by

$$T(n) = \begin{cases} \theta(n^x) & \text{when } a < b^x \quad (\text{case 1}) \\ \theta(n^x \lg n) & \text{when } a = b^x \quad (\text{case 2}) \\ \theta(n^{\log_b a}) & \text{when } a > b^x \quad (\text{case 3}) \end{cases}$$

Example(1): $T(n)=2T(n/2) + n^3$

Here $x=3$, $a=2$, $b=2$. Since $a < b^3$, case 1 applies

Therefore, $T(n)=\theta(n^3)$

Example(2): $T(n)=4T(n/2) + n^2$

$x=2$, $a=4$, $b=2$ Since $a = b^2$, case 2 applies

Therefore, $T(n)=\theta(n^2 \lg n)$

Example(3): $T(n)=7T(n/2) + n^2$

$x=2$, $a=7$, $b=2$ Since $a > b^2$, case 3 applies

Therefore, $T(n)=\theta(n^{\log_2 7})=\theta(n^{\lg 7})$

21

Theorem#2

Proof Outline

- The recurrence

$$T(1) = c$$

$$T(n) = a T(n/b) + cn^x, \quad \text{for } n > 1$$

can be solved by substitution method.

- It can be shown that after k substitutions the result is given

$$T(n) = a^k T(n/b^k) + cn^x [1 + (a/b^x) + (a/b^x)^2 + \dots + (a/b^x)^{k-1}]$$

The term $T(n/b^k)$ would reduce to base case $T(1)$ when $n/b^k = 1$, or $b^k = n$.

Taking logarithm with base b , $k = \log_b n$. Substituting for k in the summation:

$$T(n) = a^{\log_b n} T(1) + cn^x [1 + (a/b^x) + (a/b^x)^2 + \dots + (a/b^x)^{\log_b n - 1}]$$

- The geometric series has **geometric ratio** a/b^x . By considering the asymptotic behavior of the series it can be shown that

$$T(n) = \theta(n^x), \quad \text{when } a < b^x$$

$$T(n) = \theta(n^x \log n), \quad \text{when } a = b^x$$

$$T(n) = \theta(n^{\log_b a}), \quad \text{when } a > b^x$$

22

Using Theorems

Master Theorem

Let $a \geq 1$ and $b > 1$ be constants, then the recurrence

$$T(n) = aT(n/b) + f(n)$$

has the solution:

$$(1) \quad T(n) = \theta(n^{\log_b a}) \quad \text{when } f(n) = O(n^{\log_b a - \epsilon}) \text{ for some } \epsilon > 0$$

$$(2) \quad T(n) = \theta(n^{\log_b a} \lg n) \quad \text{when } f(n) = \theta(n^{\log_b a})$$

$$(3) \quad T(n) = \theta(f(n)) \quad \text{when } f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ for some } \epsilon > 0 \\ \text{provided also that } af(n/b) \leq c \cdot f(n) \text{ for some } c < 1 \text{ and large } n$$

▪ The **Master Theorem** provides a generalized solution for the **divide-and-conquer algorithms**.

The cost function can be **polynomial term** or **logarithmic function**

23

Using Master Theorem

Constraints

➤ In Master Theorem, the cost function $f(n)$ is compared with the function $n^{\log_b a}$

Depending on the outcome, the **larger** of the two functions provides the solution, subject to some additional constraints.

➤ The constraint is that the function $f(n)$ and $n^{\log_b a}$ should not be **simply larger or smaller asymptotically**, but should grow faster or slower by a **polynomial factor** n^ϵ , where ϵ is some arbitrary positive constant. Accordingly, in using Master Theorem the following conditions are examined:

Case 1: If $f(n) = O(n^{\log_b a - \epsilon})$, $f(n)$ grows slower than $n^{\log_b a}$ by a factor of n^ϵ , then solution of the recurrence is

$$T(n) = \theta(n^{\log_b a})$$

Case 2: If $f(n) = \theta(n^{\log_b a})$, i. e. $f(n)$ grows as fast as $n^{\log_b a}$, then solution of the recurrence is

$$T(n) = \theta(n^{\log_b a} \lg n)$$

Case 3: If $f(n) = \Omega(n^{\log_b a + \epsilon})$, i. e. $f(n)$ grows faster than $n^{\log_b a}$ by a factor of n^ϵ , and in addition $f(n/b) \leq c \cdot f(n)$ for some $c < 1$, then solution of the recurrence is

$$T(n) = \theta(f(n))$$

24

Using Master Theorem-I

Here are some typical examples of the use of Master Theorem for solving recurrences for the divide-and-conquer problems.

Example(1): $T(n)=4T(n/2) + n$

Here $a=4$, $b=2$, $f(n)=n$

Consider $n^{\log_b a - \epsilon} = n^{\log_2 4 - \epsilon} = n^{2-\epsilon}$. Take $\epsilon = 0.5$ (ϵ can be some *arbitrary* small constant)

Since $f(n) = n$ **grows slower than** $n^{\log_b a - \epsilon} = n^{1.5}$, it follows that $f(n) = O(n^{\log_b a - \epsilon})$

This is case I of Master Theorem. Thus, $T(n) = \theta(n^{\log_b a}) = \theta(n^{\log_2 4}) = \theta(n^2)$

Example(2): $T(n)=T(n/2) + 1$

Here $a=1$, $b=2$, $f(n)=1$

Consider $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$

Since $f(n) = 1$ **grows as fast as** $n^{\log_b a} = 1$, it follows that $f(n) = \theta(n^{\log_b a})$

This is case 2 of Master Theorem. Therefore, $T(n) = \theta(n^{\log_b a} \cdot \lg n) = \theta(n^{\log_2 1} \cdot \lg n) = \theta(\lg n)$

Example(3): $T(n)=T(n/3) + n$

Here $a=1$, $b=3$, $f(n)=1$

Consider $n^{\log_b a + \epsilon} = n^{\log_3 1 + \epsilon} = n^\epsilon$. Take $\epsilon=0.5$

Since $f(n) = n$ **grows faster than** $n^{\log_b a + \epsilon} = n^{0.5}$, it follows that $f(n) = \Omega(n^{\log_b a + \epsilon})$

Further, $af(n/b) < c \cdot f(n)$ if $1/(n/3) < c \cdot n$ i.e, $n/3 < c \cdot n$ for some c . This is true if $c=1/4$

This is case 3 of Master Theorem. Therefore, $T(n) = \theta(f(n)) = \theta(n)$

25

Using Master Theorem-II

Example(4): $T(n)=3T(n/4) + n \lg n$

Here $a=3$, $b=4$, $f(n)=n \lg n$

Consider $n^{\log_b a + \epsilon} = n^{\log_4 3 + \epsilon} = n^{0.793 + \epsilon}$. Take $\epsilon=0.207$

Since $f(n) = n \lg n$ **grows faster than** $n^{\log_b a + \epsilon} = n$, it follows that $f(n) = \Omega(n^{\log_b a + \epsilon})$

Further, $af(n/b) < c \cdot f(n)$ if $3(n/4) \lg(n/4) < c \cdot n \lg n$ some $c=3/4$.

This is case 3 of Master Theorem. Therefore, $T(n) = \theta(f(n)) = \theta(n \lg n)$

Example(5): The Master Theorem cannot be used to obtain solution to certain problems. Consider the recurrence

$$T(n)=4T(n/4) + n \lg n$$

In this case $a=4$, $b=4$, $f(n)=n \lg n$

Now, $n^{\log_b a + \epsilon} = n^{\log_4 4 + \epsilon} = n^{1+\epsilon}$

It will be seen that for **any positive ϵ** , $f(n)=n \lg n$ **does not grow faster than** $n^{1+\epsilon}$

It follows that case 3 of Master Theorem does not apply. Thus, this example cannot

Be solved by applying Master Theorem

26

Recurrences

Characteristic Equation

The algorithms for some problems can be expressed in terms of **linear homogenous relation** of the type

$$T(n) = a_{n-1}T(n-1) + a_{n-2}T(n-2) + \dots + a_1T(1).$$

- In order to solve the recurrence, the relation is usually converted into an algebraic equation. This equation is called **characteristic equation**.

- A simpler form of homogenous recursion is

$$T(n) = aT(n-1) + bT(n-2)$$

where a and b are constants. The boundary conditions for the recurrence might be

$$T(0)=0, T(1)=1$$

- The above recurrence can be solved in a closed form by converting into an algebraic **quadratic equation**.

27

Using Characteristic Equation

Procedure

The solution to the recurrence

$$T(n) = aT(n-1) + bT(n-2), \quad T(0)=0, T(1)=1$$

is obtained in a closed form by the following procedure:

Step #1: Let $T(n) = x^n$, where x is some variable

Plugging into the recurrence relation

$$x^n = a.x^{n-1} + b.x^{n-2}$$

Dividing with x^{n-2} and simplifying

$$x^2 - a.x - b = 0$$

Step #2: The quadratic equations has two roots. If x_1 and x_2 are the **distinct roots** then

$$x_1 = (a + \sqrt{a^2 + 4b}) / 2$$

$$x_2 = (a - \sqrt{a^2 + 4b}) / 2$$

Step #3: A general solution of recurrence is **linear combination of x_1^n and x_2^n** , as follows:

$$T(n) = c_1.x_1^n + c_2.x_2^n$$

where c_1 and c_2 are constants, which are obtained by applying the boundary conditions,

$$T(0)=0 \text{ and } T(1)=1$$

28

Using Characteristic Equation

Example The *Fibonacci Sequence* is a homogenous recurrence, which can be solved by using the method of characteristic equation. The recurrence has the form:

$$T(n) = T(n-1) + T(n-2) \text{ for } n > 1, \quad T(0)=0, T(1)=1$$

- Let $T(n) = x^n$
- Plugging into the relation and simplifying

$$x^2 - x - 1 = 0$$
- Solving the equation

$$x_1 = (1 + \sqrt{5}) / 2$$

$$x_2 = (1 - \sqrt{5}) / 2$$
- A general solution of recurrence is

$$T(n) = c_1 \cdot x_1^n + c_2 \cdot x_2^n$$
- Using the boundary conditions, $T(0)=0$ and $T(1)=1$

$$n=0, \quad c_1 + c_2 = T(0) = 0$$

$$n=1, \quad c_1 \cdot x_1 + c_2 \cdot x_2 = T(1) = 1$$
- Solving for c_1 and c_2

$$c_1 = 1/\sqrt{5}, \quad c_2 = -1/\sqrt{5}$$
- The recurrence has the solution

$$T(n) = (x_1^n - x_2^n) / \sqrt{5}$$

$$= [(1 + \sqrt{5}) / 2]^n - [(1 - \sqrt{5}) / 2]^n / \sqrt{5}$$

29

Induction Method

Procedure

- The *induction method* is commonly used to establish the *validity of some known recurrence*. It involves following steps
 - First it is *proved* that the recurrence $T(n)$ is valid for *boundary conditions*.
 - Next, it is *assumed* that $T(n)$ is true for some value $k < n$
 - Finally, using the basic definition and assumption at last step, it is *proved* that relation $T(n)$ also holds true for $2k$
- If the above conditions are satisfied then recurrence holds true for all n

30

Using Induction Method

Example: Using induction method it can be proved that recurrence

$$T(n) = \begin{cases} 1, & n = 2 \\ 2T(n/2) + 2 & n > 2 \end{cases}$$

has the solutions $T(n) = (3/2)n - 2$.

Step #1: $T(2) = (3/2)2 - 2 = 1$. True for base case

Step #2: Assume the solution is true for $n=k$,
 $T(k) = (3/2)k - 2 \dots\dots\dots (1)$

Step #3: Consider $T(2k)$.

$$\begin{aligned} T(2k) &= 2T(2k/2) + 2, \text{ by definition} \\ &= 2((3/2)k - 2) + 2, \text{ by assumption at (1)} \\ &= (3/2)(2k) - 2 \end{aligned}$$

It follows that the relation is true for $n=2k$. Therefore, it is true for all n